

תרגול מס' 10

- ❖ python - הכרת השפה וסביבת העבודה
- ❖ משתנים - טיפוסים משתנים, השמות ופעולות אריתמטיות
- ❖ מחרוזות
- ❖ מבני בקרה - if, while-loops



שפת התכנות פייתון

- פייתון היא שפת תכנות שזוכה לפופולריות רבה.
- קיימת קהילת מתכנתים גדולה המשתמשת בה ומגוון רב של חבילות הזמינות לכל שימוש אפשרי.
- קלה לשימוש עבור פיתוח ובחינה של קוד חדש.
- עבודה עם interpreter (הרחבה בשקופית הבאה).
- הסינטקס של השפה מאוד פשוט, ואף דומה לשפה האנגלית.
- היא תומכת במגוון רחב של מערכות הפעלה וקוד הנכתב בה יכול, במקרים רבים, לעבוד על מערכות הפעלה שונות ללא צורך בהתאמה.
- היא חינומית ו-Open Source (קוד פתוח).

Python

```
1 print("hello world!")
2
3
```

C

```
1 #include <stdio.h>
2 int main(){
3     printf("hello world!\n");
4     return 0;
5 }
6
```

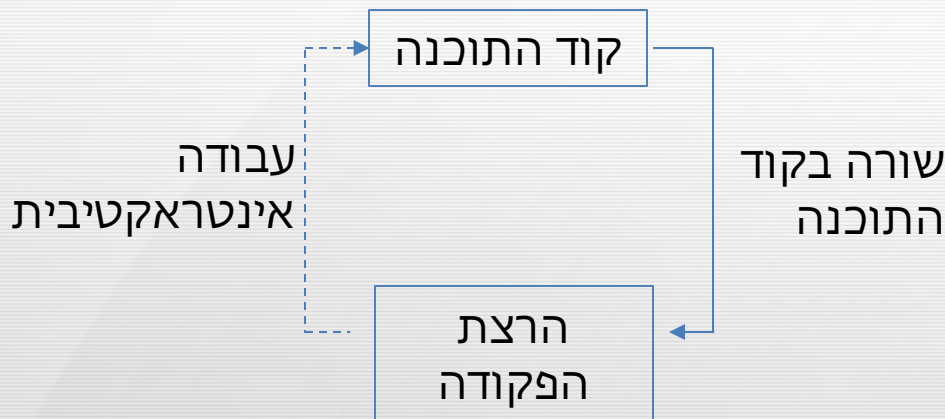
- אין צורך לנהל זיכרון בצורה ידנית. הקצאות ושחרור זיכרון מתבצעות אוטומטית.

Interpreter vs. Compiler

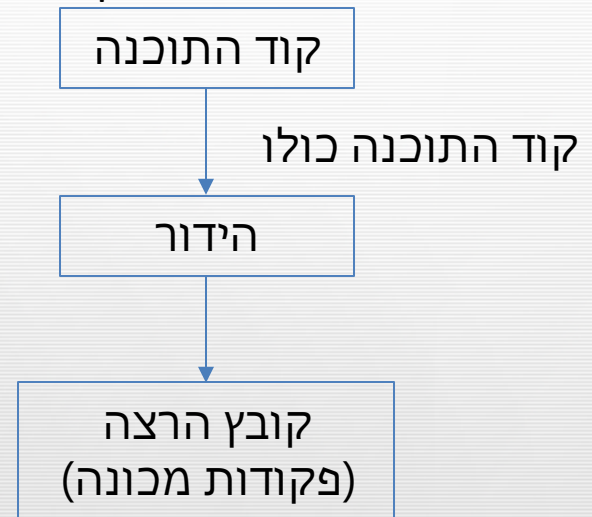
- compiler מתרגם את כל התוכנית לקוד מכונה יעיל (קובץ הרצה), שניתן להפעיל ולהריץ ישירות על ידי המעבד.
- Interpreter מבצע שורה אחר שורה בתוכנית, בצורה אינטראקטיבית.
- זמן ההרצה של תכנית שהודרה עם compiler בדרך כלל מהיר יותר מאשר תכנית העובדת עם interpreter.

כבר למדנו על שפה אחת שעובדת עם interpreter - bash

עבודה עם interpreter



עבודה עם compiler





הרצת קוד פייתון

- תחילה, יש להתקין פייתון על המחשב (המחיר של interpreter ...)

- בקורס נעבוד עם python 3.6 (כל גרסה 3.x תתאים)

- בשרת ה-CSL3 יש להשתמש בפקודה **python3.6** (כבר מותקן)

- ניתן להריץ תוכנית בפייתון בכל אחת מהדרכים הבאות:

1. הפעלה אינטראקטיבית של python (אפשר בשורת הפקודה) והזנה של פקודות שורה-שורה

- יציאה ע"י Ctrl+d (EOF) או הזנת הפקודה `exit()`

```
[mtmchk@csl3 ~]$ python3.6
Python 3.6.3 (default, Jan 27 2019, 09:41:42)
[GCC 4.8.5 20150623 (Red Hat 4.8.5-28)] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World!")
Hello World!
>>> █
```

הרצת קוד פייתון

2. הרצה של תסריט: יצירת קובץ טקסט עם קוד פייתון ("תסריט") והרצתו ע"י הרצת הפקודה

`python hello.py`

```
[mtmchk@csl3 ~]$ python hello.py
Hello, World!
[mtmchk@csl3 ~]$
```

hello.py:

```
print("Hello, World!")
```

3. הרצת תסריט כ-"קובץ הרצה" (ב-unix): הפנייה לפייתון בראש התסריט ע"י שורה שמתחילה ב-`#!/ (shebang)` והרצה כפקודה תוך מתן הרשאות ריצה ע"י `chmod`

```
[mtmchk@csl3 ~]$ cat hello.py
#!/usr/local/bin/python3.6

print("Hello, World!")

[mtmchk@csl3 ~]$ chmod u+x hello.py
[mtmchk@csl3 ~]$ ./hello.py
Hello, World!
[mtmchk@csl3 ~]$
```

– תוך מתן הרשאות ע"י `chmod`

– עובד בצורה דומה לכל interpreter

עבודה עם פייתון בסביבת עבודה (IDE)

- ניתן להתקין תוסף לVSCode המאפשר עבודה עם פייתון.

– יתרון של VSCode בעבודה על פרויקטים מרובי שפות – מאפשר עבודה עם כמה שפות בסביבת עבודה אחת.



- ישנן סביבות פיתוח פופולריות נוספות שניתן להשתמש בהן, למשל PyCharm (חינמית לסטודנטים בטכניון)

משתנים

- ❖ משתנים ב-python3
- ❖ טיפוסים משתנים
- ❖ השמות ושינוי ערכי משתנים
- ❖ פעולות אריתמטיות

משתנים מספריים

- ניתן ליצור משתנה ב-python על ידי השמה ישירה:

```
> <varname> = <value>
```

```
> x = 7
```

- **אין צורך להכריז** על משתנים ב-python, לאחר ביצוע השמה למשתנה הוא מוגדר אוטומטית.
- למשתנים אין טיפוס קבוע – רק לערכים.
- ניתן **להחליף** ערך של משתנה על ידי השמה נוספת.

```
> x = 7 # Create a new variable with the value 7
```

```
> x = 5 # Change the value of x to 5
```

הערות בפייתון
מתחילות בתו #
ונמשכות עד סוף
השורה

טיפוסי המשתנים בפייתון

- טיפוס המשתנה נקבע על פי הערך שנתנו לו בהשמה.
- ניתן לקבל את הטיפוס של המשתנה באמצעות הפקודה `type()`

```
> x = 7  
> type(x)  
<class 'int'>
```

x הוא מטיפוס int כי
יצרנו את x עם הערך 7

```
> x = 3.14  
> type(x)  
<class 'float'>
```

עכשיו x הוא מטיפוס float
כי שמנו בו מספר ממשי –
3.14

- הפונקציה `type()` יכולה לפעול גם על ערכים ולא רק על משתנים.
– לדוגמא:

```
> type(56.5555)  
<class 'float'>
```

משתנים והשמות

- המתרגם (interpreter) פועל באופן הבא:

1. מחשב את תוצאת הביטוי

2. מבצע השמה של תוצאת הביטוי אל תוך המשתנה

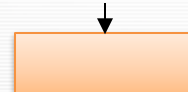
חלק שמאל של ההשמה
חייב להיות משתנה.

משתנה



ביטוי (expression)

=



חלק ימין יכול להיות כל
ביטוי הניתן לחישוב.

> n = 10

> y = (35 + 99) * 6

> 56 = y * 7

שגיאה ! צד שמאל הוא
לא משתנה

- שם המשתנה – רצף של תווים וספרות, חייב להתחיל בתו.

משתנים והשמות - דוגמאות

```
> n = 2 * 5
```

```
> print(n)
```

```
10
```

הפקודה `print()` מדפיסה למסך את ערך הביטוי שבסוגריים.

- ניתן להשתמש בערכו של משתנה כחלק מחישוב של ביטוי:

```
> m = n * 2 + 1
```

`m = 21`

- ניתן לאתחל מספר משתנים באותה השורה עם ערכים שונים:

```
> a, b = 1, 2
```

`a = 1`

`b = 2`

משתנים והשמות - דוגמאות

- ניסיון לגשת לשם של משתנה אשר לא קיים יגרום **לשגיאת זמן ריצה** אשר תגרום ליציאה מהתוכנית (במצב של הרצת הקוד כתסריט), או להפסקת ריצת הקוד וחזרה לשורת הפקודה (במצב של הפעלה אינטראקטיבית של פייתון):

```
> n = 1.7456
```

```
> n = t + 5
```

```
> NameError: name 't' is not defined
```

אופרטורים אריתמטיים

- ניתן לבצע פעולות אריתמטיות פשוטות בין משתנים מספריים על ידי הפעלת האופרטור המתאים.
- האופרטורים המתמטיים המוגדרים בשפה מוצגים בטבלה:

אופרטור	שימוש	תיאור
+	$x + y$	חיבור בין שני משתנים
-	$x - y$	חיסור בין שני משתנים
*	$x * y$	הכפלת הערך של x בערך של y
**	$x ** y$	העלאת x בחזקת y
/	x / y	מחלק את x ב y במדויק
%	$x \% y$	מודולו: השארית של חלוקת x ב y
//	$x // y$	חלוקה בשלמים (הפעולה המשלימה למודולו)

פעולות אריתמטיות - דוגמאות

```
> n = 2 * 5 # n = 10
> m = n ** 2 # n squared
> print(m)
100
```

```
> print(m / n)
10
```

```
> m = 5
> n = 3
> x = m / n
> print(x)
1.6666667
```

למרות שחילקנו שני
משתנים מטיפוס int,
תוצאת החלוקה יצאה
מספר לא שלם מטיפוס
float

```
> y = m % n
> print(y)
2
```

לחלוקה בשלמים יש
להשתמש באופרטור //

```
> x = m // n
> print(x)
1
```


strings

- משתנים מטיפוס string (מחרזת) משמשים לשמירת טקסט.
- מחרזת היא אוסף סדור של תווים.
- בפייתון, מחרזות קבועות ניתנות לתיאור על ידי גרשיים כפולים או גרש בודד. אין הבדל, בחרו את הדרך הנוחה לכם.

```
> print("Hello World")  
Hello World
```

```
> s = "Hello"  
> print(s)  
Hello  
> type(s)  
<class 'str'>
```

```
> s = 'Hello'  
> print(s)  
Hello
```

H	e	l	l	o
0	1	2	3	4
-5	-4	-3	-2	-1

ניתן לגשת לתו
ספציפי במחרזת
באמצעות האינדקס
המתאים

```
> print(s[0])  
H  
> print(s[4])  
o  
> print(s[-1])  
o  
> print(s[-3])  
l
```

אינדקס -1
תמיד ייתן את
התו האחרון
במחרזת

עוד דוגמאות - strings

```
> print('Hello "World"!')  
Hello "World"!
```

```
> print("Hello \"World\\\"!")  
Hello "World"!
```

```
> s = 'Hello'  
> type(s[0])  
<class 'str'>
```

```
> s1, s2 = 'Hello', "World"  
> print(s1 + ' ' + s2)  
Hello World
```

- אם המחרוזת מכילה סוג מסוים של גרשיים בתוכה, ניתן להשתמש בסוג השני כדי לתחום אותה:
- או להשתמש ב-escaping characters:
- בפייתון אין טיפוס 'תו' ותו בודד מיוצג כמחרוזת באורך 1.
- ניתן לשרשר מחרוזות באמצעות האופרטור '+'

Strings - המשך

- מחרוזות בפייתון אינן ניתנות לשינוי – Immutable.
- לא ניתן לשנות את התווים של מחרוזות קיימת, רק **ליצור חדשה**.
- דוגמה:

```
> s = "Dog"  
> s[0] = "d"
```



TypeError: 'str' object does not support item assignment

- כן ניתן לגרום למשתנה המתייחס למחרוזת להתייחס למחרוזת חדשה:

```
> s = "Dog"  
> s = "Cat"
```



- לא ניתן לשנות את התוכן של ערכים שהם Immutable לאחר יצירתם.
- פעולות על משתנים Immutable מחזירות **ערך חדש** ואינן מעדכנות את הערך הנוכחי.

String Slicing

■ ניתן לקבל תת מחרוזת מתוך מחרוזת באמצעות slicing :

```
> s = "Some String"
```

```
> s[start:end]
```

S	o	m	e		S	t	r	i	n	g
0	1	2	3	4	5	6	7	8	9	10

הפעולה תחזיר מחרוזת **חדשה** המבוססת על התוים של המחרוזת s המקורית באופן הבא:

- מהתו המתחיל באינדקס start במחרוזת s (כולל)

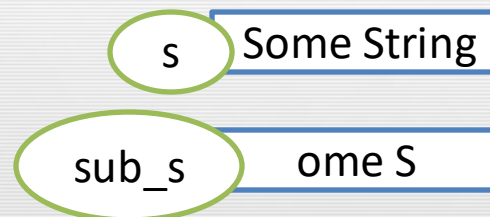
- עד התו באינדקס end במחרוזת s (לא כולל)

דוגמא:

```
> sub_s = s[1:6]
```

```
> print(sub_s)
```

```
ome S
```



String Slicing

```
> s = "Some String"  
> s[start:end:step_size]
```

- ניתן גם לציין step_size שיציין גודל קפיצה. בברירת המחדל – הקפיצות בגודל 1.

```
> s[5:7]  
St  
> s[5:7:]  
St
```

- אם לא מצוין end – אז תת-המחרוזת הנוצרת תגיע עד התו האחרון של המחרוזת המקורית:

```
> s[5:]  
String  
> s[5::2]  
Srnr
```

- אם לא מצוין start – אז תת-המחרוזת הנוצרת מתחילה מהתו הראשון של המחרוזת המקורית:

```
> s[:3]  
Som
```

פעולות נוספות על מחרוזות

- הפונקציה **len** מחזירה את אורך המחרוזת

```
> mystr = "Hello"  
> len(mystr)  
5
```

- המתודות **startswith** ו-**endswith** בודקות האם המחרוזת מתחילה או מסתיימת במחרוזת נתונה

```
> mystr = "Hello"  
> mystr.startswith("He")  
True
```

- המתודה **strip** מוחקת מהמחרוזת רווחים בהתחלה ובסוף

```
> mystr = "    Hello    "  
> mystr.strip()  
Hello
```

פעולות נוספות על מחרוזות

- המתודות **islower**, **isupper** מחזירות האם מחרוזת היא כולה אותיות גדולות או קטנות

```
> mystr = "hello"  
> mystr.islower()  
True
```

- המתודות **lower** ו-**upper** ממירות מחרוזת להיות כולה באותיות גדולות או קטנות

```
> mystr = "Hello"  
> mystr.upper()  
HELLO
```

- המתודות **isdigit**, **isalpha** בודקות האם מחרוזת היא כולה אותיות או ספרות

```
> mystr = "021334"  
> mystr.isdigit()  
True
```

פעולות נוספות על מחרוזות

- המתודה **find** מחזירה את האינדקס הראשון במחרוזת שהחל ממנו מופיעה המחרוזת הנתונה

```
> mystr = "hello"  
> mystr.find("lo")  
3
```

- המתודה **replace** מקבלת שתי מחרוזות ומחליפה כל מופע של המחרוזת הראשונה במחרוזת השנייה

```
> mystr = "Hello"  
> mystr.replace("el", "ro")  
Hrolo
```

- המתודה **isspace** בודקת אם מחרוזת כולה תווי רווח (רווח, טאב, ירידת שורה...)

```
> mystr = "  
> mystr.isspace()  
True
```

הפקודה print

- הפקודה print מאפשרת מספר שימושים מתקדמים נוחים.
- למשל, ניתן להעביר לה מספר שרירותי של ארגומנטים מופרדים בפסיק, והיא תדפיס אותם עם רווחים ביניהם:

```
> print(3, 5, 6)  
3 5 6
```

- ניתן גם לבחור מפריד אחר באמצעות התחביר הבא (שיילמד לעומק בהמשך):

```
> print(3, 5, 6, sep=";")  
3;5;6
```

- print יכולה לקבל מגוון רחב של טיפוסים:

```
> print(3, 5.4, "Cat")  
3 5.1 Cat
```

f-Strings

- בפייתון ניתן לבצע formatting של מחרוזת, כלומר השתלת ערכים בתוך מחרוזת, באמצעות התחביר הבא:

```
> x = 5
> s = f"My number is {x} and twice my number is {2*x}"
> print(s)
My number is 5 and twice my number is 10
```

- ניתן להגדיר גם את הפורמט בו יודפסו הערכים בסוגריים:

```
> pi = 3.14159265358979323846264338327950288419716939937510582097494...
> s = f"Pi equals {pi:.2f}, or more accurately {pi:.6f}"
> print(s)
'Pi equals 3.14, or more accurately 3.141593'
```

רשימות - lists

- ❖ אתחול רשימה
- ❖ גישה לאיברים
- ❖ הוספת איברים ושרשור רשימות
- ❖ מחיקת איברים
- ❖ פעולות נוספות על רשימות

lists

- list היא מבנה נתונים מסוג רשימה אשר מסוגל להכיל איברים לפי סדר, מכל טיפוס
- בשביל ליצור רשימה עם איברים ספציפיים, יש להקיפם בסוגריים מרובעים []
- list יותר דומה ל-std::vector מ-C++ מאשר לרשימה מקושרת

```
> new_list = [ 1, 2, 3, 4, 5 ]  
> print(new_list)  
[1,2,3,4,5]
```

```
> empty_list = []  
> print(empty_list)  
[]
```

```
> zeros = [0]*5  
> print(zeros)  
[0, 0, 0, 0, 0]
```

```
> mixed_types = [7, 'dog', [7], 3.14]  
> print(mixed_types)  
[7, 'dog', [7], 3.14]
```

indexing

- גישה לאיברים וביצוע slicing מתבצע באותו אופן כמו עם מחרוזות.

```
my_list = ["Baraa", "Daniel", "Ron", "Regev"]
```

```
> my_list[0]
```

```
"Baraa"
```

```
> my_list[3]
```

```
"Regev"
```

```
> my_list[4]
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

IndexError: list index out of range

```
> my_list[-1]
```

```
"Regev"
```

Slicing

```
0      1      2      3      4      5      6      7      8      9
my_list = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
> my_list[1:5] # Slicing
['b', 'c', 'd', 'e']
```

Slicing does NOT change the original list

```
> my_list
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

Slicing using an arithmetic progression

```
>>> my_list[0:9:2]
['a', 'c', 'e', 'g', 'i']
```

start stop step

We can omit any of these

```
>>> my_list[0::2]
['a', 'c', 'e', 'g', 'i']
```

```
>>> my_list[1::2]
['b', 'd', 'f', 'h', 'j']
```

הוספת איברים

- ניתן להוסיף איברים לרשימה במספר דרכים
- באמצעות המתודה **append(value)** ניתן להוסיף איבר לסוף הרשימה
- באמצעות המתודה **insert(pos, value)** ניתן להוסיף איבר למיקום ספציפי

```
new_list = [ 1, 2, 3, 4, 5 ]  
print(new_list)
```

```
> [1,2,3,4,5]
```

```
new_list.append(8)  
print(new_list)
```

```
> [1,2,3,4,5,8]
```

```
new_list = [ 1, 2, 3, 4, 5 ]  
print(new_list)
```

```
> [1,2,3,4,5]
```

```
new_list.insert(1, 'one')  
print(new_list)
```

```
> [1,'one',2,3,4,5]
```

```
new_list[1] = 10
```

```
> [1,10,2,3,4,5]
```

הוספת איברים

- באמצעות הפעולה `extend(other_list)` ניתן לשרשר רשימה אחרת לסוף הרשימה.
- ניתן לשרשר רשימות גם באמצעות אופרטור `+`

```
new_list = [ 1, 2, 3, 4, 5 ]  
new_list.extend([23, 22])
```

```
[1,2,3,4,5,23,22]
```

```
new_list.append([55, 56])
```

```
[1,2,3,4,5,23,22, [55, 56]]
```

```
new_list = [1,2,3] + [4,5]
```

```
new_list += [23, 22]
```

```
[1,2,3,4,5,23,22]
```

שינוי ערכים קיימים

- שינוי ערכים ברשימה מתבצע באמצעות גישה לאיברים עם האינדקס המתאים וכתיבה מחדש לתוכם (בניגוד למחרוזות, רשימות הן Mutable):

```
new_list = [ 1, 2, 3, 4, 5 ]
```

```
new_list[3] = 77
```

```
[1, 2, 3, 77, 5]
```

הסרת איברים מרשימה

- ניתן להסיר איברים מרשימה במספר דרכים
- באמצעות המתודה **remove(value)** ניתן למחוק המופץ הראשון של האיבר value ברשימה.
- באמצעות הפקודה **del my_list[index]** ניתן למחוק מהרשימה my_list את האיבר במקום ה-index.
- המתודה **pop(index)** מוציאה את האיבר במקום ה-index ומחזירה אותו.

```
new_list = [ 1, 2, 3, 4, 2, 5 ]  
new_list.remove(2)
```

```
[ 1, 3, 4, 2, 5 ]
```

```
del new_list[2]
```

```
[ 1, 3, 2, 5 ]  
x = new_list.pop(1)
```

```
[ 1, 2, 5 ]
```

```
print(3)  
2
```

```
list_ch = [ 'a', 't', 'l', 'm']  
s = list_ch.pop()
```

```
[ 'a', 't', 'l']
```

```
print(s)  
'm'
```

העתקה של רשימות - copy

- שימו לב: ערכים שאינם מטיפוסי בסיס (טיפוסים מספריים) מאוחסנים ומועברים by reference בפייתון (כלומר, הם מתנהגים כמצביעים). בפרט, שינוי של הערך דרך משתנה אחד יגרום לו להשתנות גם בכל המשתנים האחרים שמצביעים לאותו הערך.

- לדוגמה, מה יהיה הפלט של קטע הקוד הבא?

```
> a = [1, 2, 3]
> b = a
> b[1] = 5
> print(a)
[ 1, 5, 3]
```

לחילופין, ניתן ליצור עותק בצורה מפורשת:

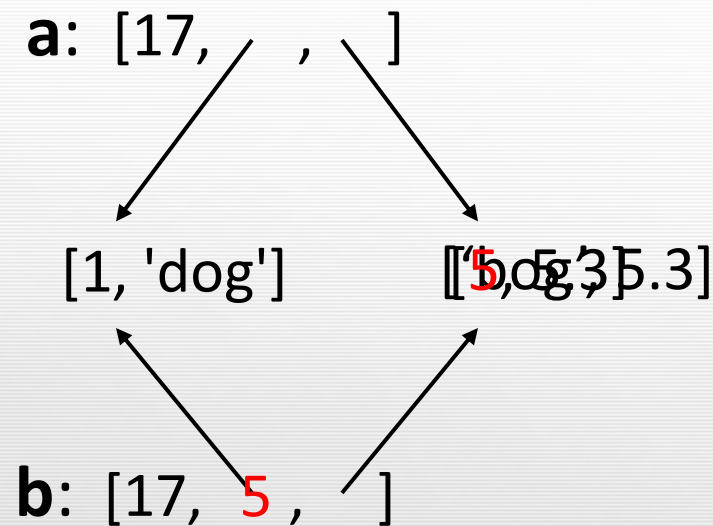
```
> b = a.copy()
> b[1] = 5
> print(a, b)
[ 1, 2, 3] [ 1, 5, 3]
```

העתקה של רשימות - deepcopy

- מה קורה ברמת כינון יותר עמוקה (רשימה של רשימות)?

```
> a = [17, [1, 'dog'], ['bog', 5.3]]  
> b = a.copy()  
> b[1] = 5  
> b[2][0] = 5
```

```
> print(b)  
[17, 5, [5, 5.3]]  
> print(a)  
[17, [1, 'dog'], [5, 5.3]]
```



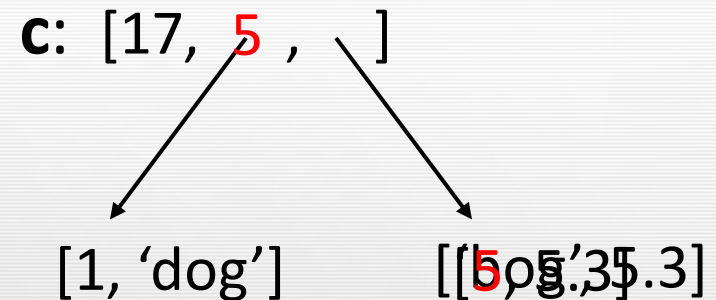
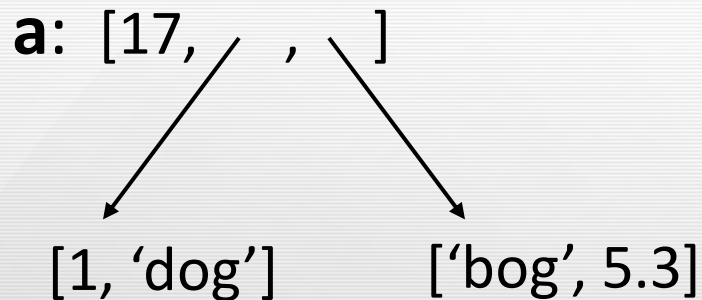
העתקה של רשימות - deepcopy

- להעתקה ברמת כינון יותר עמוקה (רשימה של רשימות) יש להשתמש ב-deepcopy:

```
> from copy import deepcopy  
> c = deepcopy(a)  
> c[1] = 5  
> c[2][0] = 5
```

```
> print(c)  
[17, 5, [5, 5.3]]  
> print(a)  
[[1, 'dog'], ['bog', 5.3]]
```

מתוך קובץ הספרייה הסטנדרטית
copy ייבא את הפונק' deepcopy
שמוגדרת בו



שני סוגי השוואות

■ בפייתון מוגדרים שני סוגי אופרטורים להשוואה:

1. אופרטור **is** שבודק זהות עצמים, האם מדובר באותו מקום בזיכרון ממש (identity)
2. אופרטור **==** שבודק שיוויון, בהתאם לטיפוס (equality)

```
> a = [1, 2, 3]
```

```
> b = a
```

```
> c = a.copy()
```

```
> a is b
```

True

```
> print(a is c, a == c)
```

False True

■ ומה לגבי טיפוסים בסיסיים?

```
> a = 3
```

```
> b = 3
```

```
> c = a
```

```
> print(a is b, a is c)
```

??

תלוי מימוש / אופטימיזציות
(תקף גם למחרוזות)

```
> a = 3
```

```
> b = a
```

```
> a = 4
```

```
> print(a is b, b)
```

False 3

מובטח!

פעולות נוספות על רשימה

- הפונקציה len מחזירה את אורך הרשימה

```
> my_list = [1, 3, 2, 0, -1, 1, 1]
> len(my_list)
7
```

- הפונקציות max ו-min מחזירות את האיבר המקסימלי והמינימלי ברשימה

```
> my_list = [1, 3, 2, 0, -1, 1, 1]
> max(my_list)
3
```

- המתודה list.count(value) מחזירה את מספר המופעים של ערך מסוים ברשימה

```
> my_list = [1, 3, 2, 0, -1, 1, 1]
> my_list.count(1)
3
```

פעולות נוספות על רשימה

- המתודה `list.index(value)` מחזירה את האינדקס **הנמוך ביותר** בו איבר נמצא ברשימה, או זורקת שגיאה אם האיבר לא נמצא ברשימה.

```
> my_list = [1, 3, 2, 0, -1, 1, 1]
> my_list.index(1)
0
```

- המתודה `list.sort()` ממיינת את הרשימה בסדר עולה (משנה את הרשימה).

```
> my_list = [1, 3, 2, 0, -1, 1, 1]
> my_list.sort()
> print(my_list)
[-1, 0, 1, 1, 1, 2, 3]
```

- הפעולה `sorted(list)` מחזירה עותק ממיון של הרשימה.

```
> my_list = [1, 3, 2, 0, -1, 1, 1]
> sorted(my_list)
[-1, 0, 1, 1, 1, 2, 3]
```

פעולות נוספות על רשימה

- המתודה `list.reverse()` תהפוך את סדר האיברים ברשימה (משנה את הרשימה).

```
> my_list = [1, 3, 2, 0, -1, 1, 1]
> my_list.reverse()
[1, 1, -1, 0, 2, 3, 1]
```

- בדומה ל-`sort, sorted` קיימת הפעולה `reversed` אשר מחזיר עותק הפוך של הרשימה.
- בדיקה האם איבר קיים ברשימה באמצעות אופרטור `in`:

```
> my_list = [1, 3, 2, 0, -1, 1, 1]
> 4 in my_list
False
```

- האופרטור `in` עובד גם לסוגי רצפים ומבני נתונים אחרים, למשל מחרוזות.

n-יות - tuples

- tuple היא רשימה שהיא immutable. כלומר, לא ניתן לשנותה לאחר שנוצרה (אבל אפשר להעתיק)
- בשביל ליצור n-יה עם איברים ספציפיים, יש להקיפם בסוגריים עגולים ()

```
> new_tuple = ( 1, 2, 3, 4, 5 )  
> print(new_tuple)  
(1,2,3,4,5)
```

```
> new_tuple[2] = 0  
Type Error: 'tuple' object does not  
support item assignment
```

```
> empty_tuple = ()  
> print(empty_tuple)  
( )
```

```
> zeros = (0, ) * 5  
> print(zeros)  
(0, 0, 0, 0, 0)
```

מקרה מיוחד: עבור tuple עם איבר בודד מציינים עם פסיק שהסוגריים תוחמים tuple והם אינם סוגריים של חישוב מתמטי

```
> mixed = (7, 'dog', [7], 3.14)  
> mytuple = (5, 3) + mixed  
> print(mytuple)  
(5,3,7,'dog',[7],3.14)
```

פעולת חיבור בין שני tuples מחזירה tuple חדש עם שרשרת התכנים שלהם

List מול Tuple

- מאחר ש-Tuple מייצג רצף שהוא Immutable, במקרים בהם נצטרך לשנות את הרצף לאחר יצירתו, נעדיף להשתמש ב-List.
- מתי בכל זאת נשתמש ב-Tuple?
 - אם נרצה לוודא שהתוכן של הרצף נשאר קבוע ואינו משתנה בטעות. לעיתים קרובות נרצה להעביר כמה ערכים יחד ממקום למקום ללא כוונה לשנותם או להוסיף/להוריד מהרצף.
 - במקרה כזה, שימוש ב-Tuple אוכף תכונה זו ומקל על הקריאות של הקוד – מדגיש שהרצף הוא Immutable.
 - מאחר שהתוכן של Tuple הוא קבוע, הוא מעט יותר יעיל בזיכרון ובזמן ריצה.
 - כפי שנלמד בהמשך, ניתן לאחסן Tuple במבנה הנתונים Set המממש קבוצה בפייתון (כמו גם במבני נתונים אחרים), בעוד שלא ניתן לאחסן List ב-Set. זה מכיוון שאם נכניס שני List שונים ואז נשנה אחד מהם להיות שווה לאחר, אנחנו נשבור את הקבוצה (שאמורה להכיל רק עותק אחד מכל ערך).

רצפים - Sequences

- מחרוזות, רשימות ו-n-יות אלו טיפוסים רצפים מובנים בפייתון
- על כולם מוגדרות פעולות len, slicing וגם שרשור, index ו-count
- מה ההבדלים ביניהם?
- בהמשך נראה כיצד סינטקס לולאות for מבוסס על רצפים

אופרטור הפיזור *

- קיים בפייתון האופרטור * (כוכבית) הפועל על טיפוסים המייצגים רצף.
- האופרטור הופך את איברי הרצף לסדרת ערכים עם פסיקים ביניהם, שניתן להשתמש בה בכל הקשר שבו פייתון מצפה לקבל סדרת ערכים עם פסיקים ביניהם. למשל:

```
> mixed = (5, 'dog', [7], 3.14)
> print(mixed)
(5, 'dog', [7], 3.14)

> mytuple = (3, mixed)
> print(mytuple)
(3, (5, 'dog', [7], 3.14))

> mytuple = (3, *mixed) # like (3, 5, 'dog', [7], 3.14)
> print(mytuple)
(3, 5, 'dog', [7], 3.14)

> mytuple = (3, *mixed) # like print(3, 5, 'dog', [7], 3.14)
> print(*mytuple)
3 5 dog [7] 3.14
```

Unpacking

- כאשר ישנו רצף עם n איברים באגף ימין של השמה, פייתון מאפשר לבצע השמה של הרצף ל- n **משתנים** שכל אחד מהם יקבל ערך אחד מהרצף. פעולה זו נקראת unpacking – פירוק של הרצף:

```
> my_tuple = (1, 2)
> a, b = my_tuple
> print(a, b)
1 2
```

- אם אנחנו מעוניינים לדלג על ערך, הקונבנציה היא לשים אותו ב-`_`, כך:

```
> my_tuple = (1, 2, 3)
> a, _, b = my_tuple
> print(a, b)
1 3
```

מבני בקרה

if statements ❖

if-else ❖

while loop ❖

משפטי if ב-python

- בפייתון משפטי תנאים מתבצעים באופן הבא:

```
if <condition>:  
  <TAB>statement1  
  <TAB>statement2  
  ...
```

אפשר גם 4 רווחים (מקובל),
חייב להיות **אחד בקובץ הקוד**

- ה-`condition` הוא כל ביטוי אשר מחזיר ערך בוליאני – לוגי מטיפוס `bool`
- ההזחה מגדירה את הסקופ של בלוק ה-`if`
- בניגוד לשפת C, בפייתון אין סגירה ופתיחה מפורשים של בלוקים (אין סגירה ופתיחה עם סוגריים מסולסלים)

Indentation!

משפטי if ב-python

• דוגמאות למשפטי if:

```
if 32 % 8 == 0: # the remainder of 32 divided by 8
    print ("32 is divisible by 8")
    print ("32 is also divisible by 2 and 4")
```

32 is divisible by 8

32 is also divisible by 2 and 4

```
x = 9
y = 7
if x > y:
    print("x is greater than y")
    x -= 1
print(x)
```

x is greater than y

8

if-else

- ניתן לכתוב ביטויי if-else בפייתון:

```
if x > y:  
    print("x is greater than y")  
else:  
    print("y is greater than or equal to x")
```

- במידה ויש יותר מ-2 אפשרויות ניתן לרשום ביטויי if-elif-else

```
if x > y:  
    print("x is greater than y")  
elif x == y:  
    print("x is equal to y")  
else:  
    print("y is greater than x")
```

- מותר גם יותר מ-elif אחד.
- ה-else הסופי הוא אופציונלי.
- קיים גם אופרטור טרנארי:

```
print("x is greater than y") if x > y else print("y is greater than or equal to x")
```

אופרטורים לוגיים וקבועי Boolean

- באמצעות האופרטורים **and**, **or**, **not** ניתן לשלב ביטויי תנאי מורכבים:

```
x = 3
y = -3
if x >= 2 and y <= -2:
    print("True")
if x > 3 or y < -3:
    print("True")
if not x == y: # also: x != y
    print("True")

if x > y or not (x >= 3 and y <= -3):
    print("True")
```

```
x = True
y = False
x and y
> False
```

```
type(x)
<class 'bool'>
```

- סדר הקדימויות של האופרטורים:

1. not
2. and
3. or

- ייצוג משתני Boolean בפייתון:

- "אמת" מיוצג על ידי True
- "שקר" מיוצג על ידי False
- שימו לב לאות הראשונה הגדולה

לולאות while

- בפייתון משפטי while מתבצעים באופן הבא:

```
while <expression>:  
    statement1  
    statement2  
    ...
```

- ה-expression הוא כל ביטוי אשר

מחזיר ערך Boolean

- ההזחה מגדירה את הסקופ של בלוק ה-while

- ניתן להשתמש בפקודות break ו-continue בתוך לולאות while

כמו בשפת C

- דוגמה:

```
count = 0  
while count < 3:  
    print("count:", count)  
    count += 1  
print("Done")
```

```
count: 0  
count: 1  
count: 2  
Done
```